

# Wireless PSI & ASI

**FREE Technical Subject Session**

|           |          |                                         |
|-----------|----------|-----------------------------------------|
| 19-1-2026 | સોમવાર   | Number Systems & Boolean Logic          |
| 20-1-2026 | મંગળવાર  | Logic Gates & Combinational Logic       |
| 21-1-2026 | બુધવાર   | LAN Technologies                        |
| 22-1-2026 | ગુરુવાર  | IP Addressing                           |
| 23-1-2026 | શુક્રવાર | Semiconductor Physics, Diodes & Biasing |
| 24-1-2026 | શનિવાર   | Transistors                             |



કોઈપણ કોર્સ કે બુક ખરીદવા માટે આજે જ  
GYANLIVE એપ ડાઉનલોડ કરો.



**89806 77555**  
**84696 77555**

: Join Us :



# NUMBER SYSTEM & BOOLEAN LOGIC

DIGITAL ELECTRONICS

Gujarat's  
**No.1** Live  
Courses  
App



# Wireless PSI & ASI (TO) માટે

**Part B : 120 ગુણ ટેકનિકલ**

સંપૂર્ણ સિલેબસ, pdf મટીરીયલ્સ અને ટેસ્ટ્સ સાથે...

## Qualification

B.E. | B.Tech in E.C. | Elect | E.T.  
I.T. | ICT | C.E. | C.S.

LIVE

With Book

**₹ 6999/-**

કોઈપણ કોર્સ કે બુક ખરીદવા માટે  
આજે જ GYANLIVE એપ ડાઉનલોડ કરો.



**89806 77555**  
**84696 77555**



# Introduction To Number System

- The concept of a Number System is the mathematical notation for representing and expressing numbers using a defined set of digits or other symbols in a consistent manner.

$$(101)_8 \Rightarrow (101)_3 \Rightarrow (101)_{16}$$

- In digital electronics and computer systems, number systems are essential because they provide a standardized way to store, process, and communicate numerical information

$$(101)_{10} \leftarrow \text{Decimal}$$

- Each number system is characterized by its base (radix), which indicates the total number of distinct digits used, including zero.

# Elements of a Number System

- **Base (Radix):** The number of unique digits in the system.
- **Digits:** Symbols used to represent numbers.
- **Positional Value:** The value of a digit depends on its position and the base of the system.

Ex: 932  
10

$$\begin{array}{c} 9 \\ \times 100 \\ \hline \end{array} + \begin{array}{c} 3 \\ \times 10 \\ \hline \end{array} + \begin{array}{c} 2 \\ \times 1 \\ \hline \end{array} \Rightarrow 932$$

# Number Systems Commonly Used

In mathematics and computing, we primarily deal with four types of systems based on their base (the number of unique digits used).

| Number System | Base (Radix) | Digits Used                  | Primary Use                |
|---------------|--------------|------------------------------|----------------------------|
| Binary ✓      | 2            | 0, 1                         | Computers (logic gates)    |
| Octal ✓       | 8            | 0, 1, 2, 3, 4, 5, 6, 7       | Computing (compact binary) |
| Decimal ✓     | 10           | 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 | Everyday life (counting)   |
| Hexadecimal ✓ | 16           | 0-9 and A-F                  | Color codes, MAC addresses |

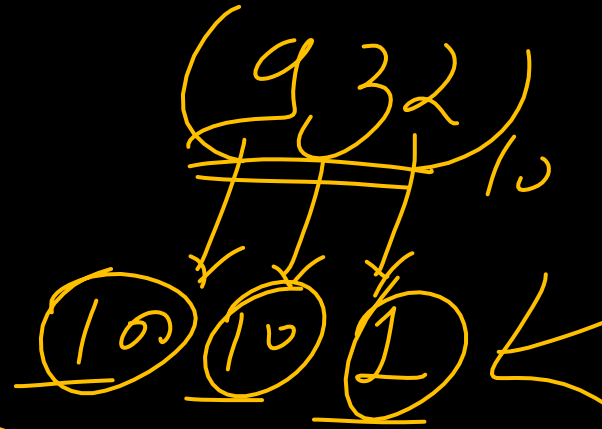
10 → [A] 11 → [B] 12 → [C] — — — (15) → [F]

# Positional Value (Positional weight)

- The most important rule in modern number systems is Place Value.
- The value of a digit depends on its position. For example, in the decimal

number 345:

- The 5 is in the "ones" place ( $5 \times 10^0$ ).
- The 4 is in the "tens" place ( $4 \times 10^1$ ).
- The 3 is in the "hundreds" place ( $3 \times 10^2$ ).



- This logic applies to all bases. ***In binary, the places are powers of 2 (1, 2, 4, 8...) instead of powers of 10.***

$2^0, 2^1, 2^2, \dots$

# Plotting Decimal Number

- Let's take decimal number = 1638
- Formation of decimal number:

|     |        |        |        |        |        |
|-----|--------|--------|--------|--------|--------|
|     |        |        |        |        |        |
| ... | $10^4$ | $10^3$ | $10^2$ | $10^1$ | $10^0$ |
| ... | 10000  | 1000   | 100    | 10     | 1      |

$$\begin{aligned} & (1 \times 1000) + (6 \times 100) + (3 \times 10) + (8 \times 1) \\ \Rightarrow & \underline{1000} + \underline{600} + \underline{30} + \underline{8} \\ \Rightarrow & \underline{1638} \end{aligned}$$

# Decimal to Binary Conversion

► To convert a decimal number (Base 10) into a binary number (Base 2), the most common method is **Successive Division by 2**.

► Essentially, you keep dividing the number by 2 and tracking the remainders until you reach zero.

## ► Procedure:

- ✓ Divide the decimal number by 2.
- ✓ Record the remainder (0 or 1).
- ✓ Divide the quotient again by 2.
- ✓ Repeat until the quotient becomes zero.
- ✓ The binary equivalent is obtained by reading the remainders from bottom to top.



# Example: Converting $(37)_{10}$ To Binary

- Divide the number repeatedly by 2 and note the remainders:

| Division Step | Quotient | Remainder |
|---------------|----------|-----------|
| $37 \div 2$   | 18       | 1 (LSB)   |
| $18 \div 2$   | 9        | 0         |
| $9 \div 2$    | 4        | 1         |
| $4 \div 2$    | 2        | 0         |
| $2 \div 2$    | 1        | 0         |
| $1 \div 2$    | 0        | 1 (MSB)   |

Handwritten notes illustrating the conversion process:

- Arrows pointing from the remainders in the table to the binary result:  $2121001 \rightarrow 2121001$  (with a circled '2121' and '001' below it).
- A large circle containing the text "Last most Step" with an arrow pointing to the final remainder (1) in the table.

- Read the remainders from bottom to top (MSB to LSB).

Result:  $(37)_{10} = (100101)_2$  [0-1] bit

# ASCII Value

- ASCII stands for American Standard Code for Information Interchange.
- It is a character-encoding standard that assigns numeric values to letters, digits, punctuation symbols, and control characters.
- Computers and digital systems represent and exchange textual data in a uniform and consistent manner.

- Basic Concept:

1 2 3 4 5 6 7 ← bits

- ✓ ASCII uses 7 bits (binary digits) to represent each character.
- ✓ This allows 128 unique codes, ranging from 0 to 127.
- ✓ Each character (such as a letter or symbol) is mapped to a unique decimal value called its ASCII value.

# Example of ASCII Values:

◆ **Digits (0-9):** 48-57

◆ **Alphabets:**

✓ Uppercase letters (A-Z): 65 - 90

✓ Lowercase letters (a-z): 97 - 122

✓ Difference of ASCII Value of Small letter with capital letter is: 32.

✓ **Example:** ASCII value of 'A' is 65 and that of 'a' is 97.

$$\text{So, } 97 - 65 = 32$$

ASCII of P  
ASCII of p  
32

# Extended ASCII

- Extended ASCII uses 8 bits, allowing 256 characters (0–255).
- It includes additional symbols, accented letters, and graphical characters.
- It is not fully standardized and may vary across systems.

# Binary To Decimal Conversion

- Binary to decimal conversion is done using the positional weight method.
- Which is a straightforward process where each digit (bit) is multiplied by a power of 2 based on its position.
- **Procedure:**
  - ✓ Assign weights to each bit based on powers of 2, starting from the rightmost digit (least significant bit) to leftwards with  $2^0$ ,  $2^1$ ,  $2^2$ ...
  - ✓ Multiply each bit by its corresponding weight (power of 2).
  - ✓ Add all the products to obtain the decimal value.

# Example: Converting $(100101)_2$ To Decimal

For given binary number  $(100101)_2$  :

- Write each bit with its corresponding power of 2, starting from the rightmost bit:

$$= (1 \times 2^5) + (0 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$$

$$= (1 \times 32) + (0 \times 16) + (0 \times 8) + (1 \times 4) + (0 \times 2) + (1 \times 1)$$

$$= 32 + 0 + 0 + 4 + 0 + 1$$

$$= (37)_{10}$$

base 10  $\rightarrow$  base 8

## Decimal to Octal Conversion

- ◇ Decimal to octal conversion uses the repeated division-by-8 method, similar to decimal-to-binary conversion.
- ◇ **Procedure:**
  - ✓ Divide the decimal number by 8.
  - ✓ Record the remainder (0-7).
  - ✓ Continue dividing the quotient by 8 until it becomes zero.
  - ✓ Read the remainders from bottom to top to get the octal equivalent.

Example: Convert  $(125)_{10}$  to Octal

| Division Step                 | Quotient  | Remainder |
|-------------------------------|-----------|-----------|
| $125 \div 8$                  | <u>15</u> | <u>5</u>  |
| <u><math>15 \div 8</math></u> | <u>1</u>  | <u>7</u>  |
| <u><math>1 \div 8</math></u>  | 0         | <u>1</u>  |

Reading the remainders from bottom to top:  ~~$(125)_{10}$~~  =  $(175)_8$

# Decimal To Hexadecimal Conversion

- This conversion is performed using the repeated division-by-16 method.
- **Procedure:**
  - ✓ Divide the decimal number by 16.
  - ✓ Record the remainder (0–15).
  - ✓ Replace remainders 10–15 with A–F.
  - ✓ Read the remainders from bottom to top to obtain the hexadecimal number.

# Example: Convert $(254)_{10}$ To Hexadecimal

| Division Step                   | Quotient  | Remainder           |
|---------------------------------|-----------|---------------------|
| <u><math>254 \div 16</math></u> | <u>15</u> | <u>14</u> (i.e., E) |
| <u><math>15 \div 16</math></u>  | <u>0</u>  | <u>15</u> (i.e., F) |

- Reading the remainders from bottom to top:  $(254)_{10}$  =  $(FE)_{16}$

## Binary to Octal Conversion

→ [0-7] ⇒ 3 bit

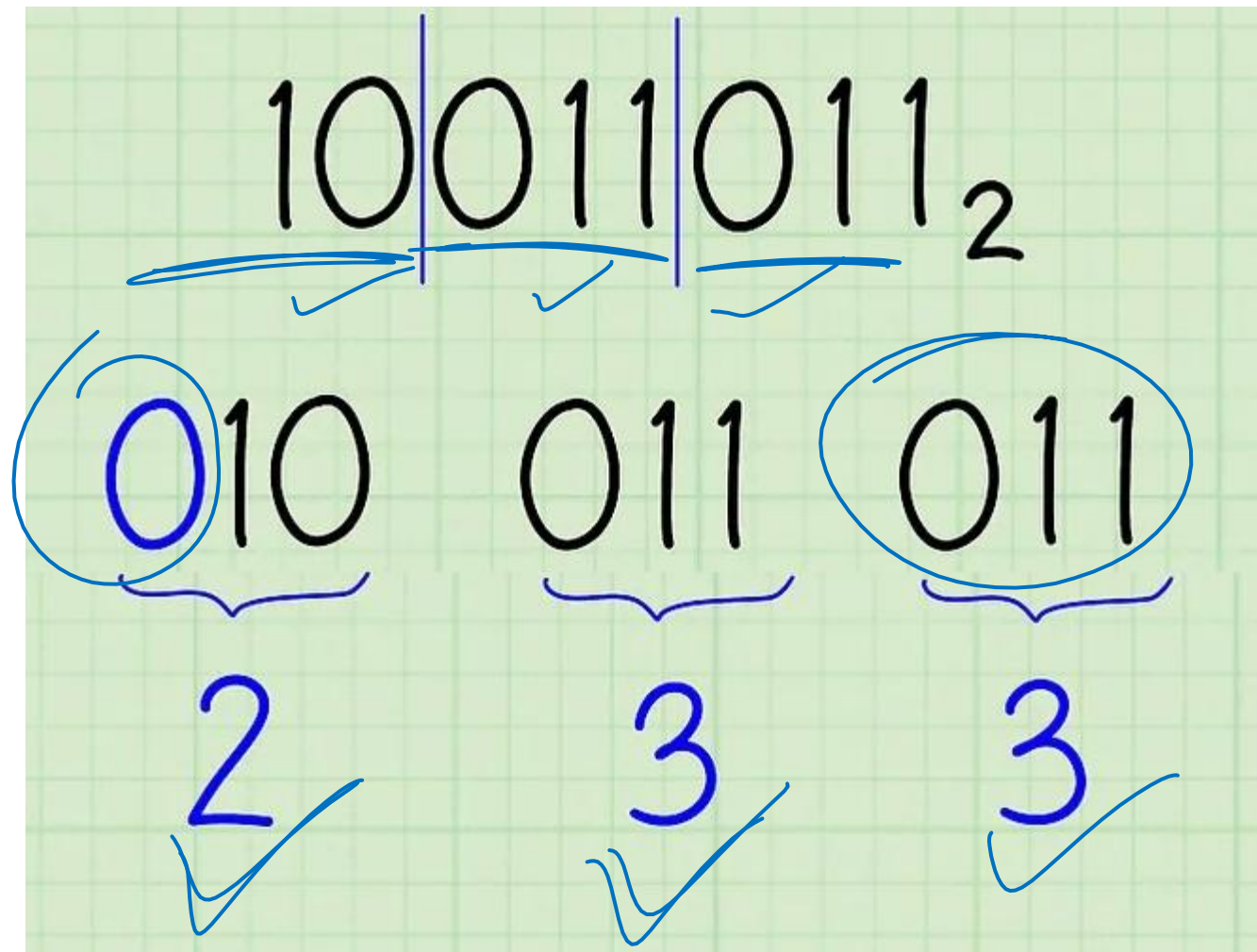
- Binary numbers are converted to octal by grouping bits in sets of three, starting from the right.
- **Procedure:**
  - ✓ Group the binary digits into 3-bit groups.
  - ✓ Convert each group into its octal equivalent using a binary-octal table.

# Binary to Octal Table

| Binary<br>(Base 2) | Octal<br>(Base 8) | Binary<br>(Base 2) | Octal<br>(Base 8) |
|--------------------|-------------------|--------------------|-------------------|
| <u>000</u>         | 0                 | <u>100</u>         | 4                 |
| <u>001</u>         | 1                 | <u>101</u>         | 5                 |
| <u>010</u>         | 2                 | <u>110</u>         | 6                 |
| <u>011</u>         | 3                 | <u>111</u>         | 7                 |



$$(10011011)_2 \Rightarrow (?)_8$$



$$(233)_8$$

→ [0-15] => 4 bits

# Binary To Hexadecimal Conversion

- Binary to hexadecimal conversion is done by grouping bits in sets of four.

- **Procedure:**

- ✓ Group binary digits into 4-bit groups from the right.
- ✓ Convert each group into its hexadecimal equivalent.

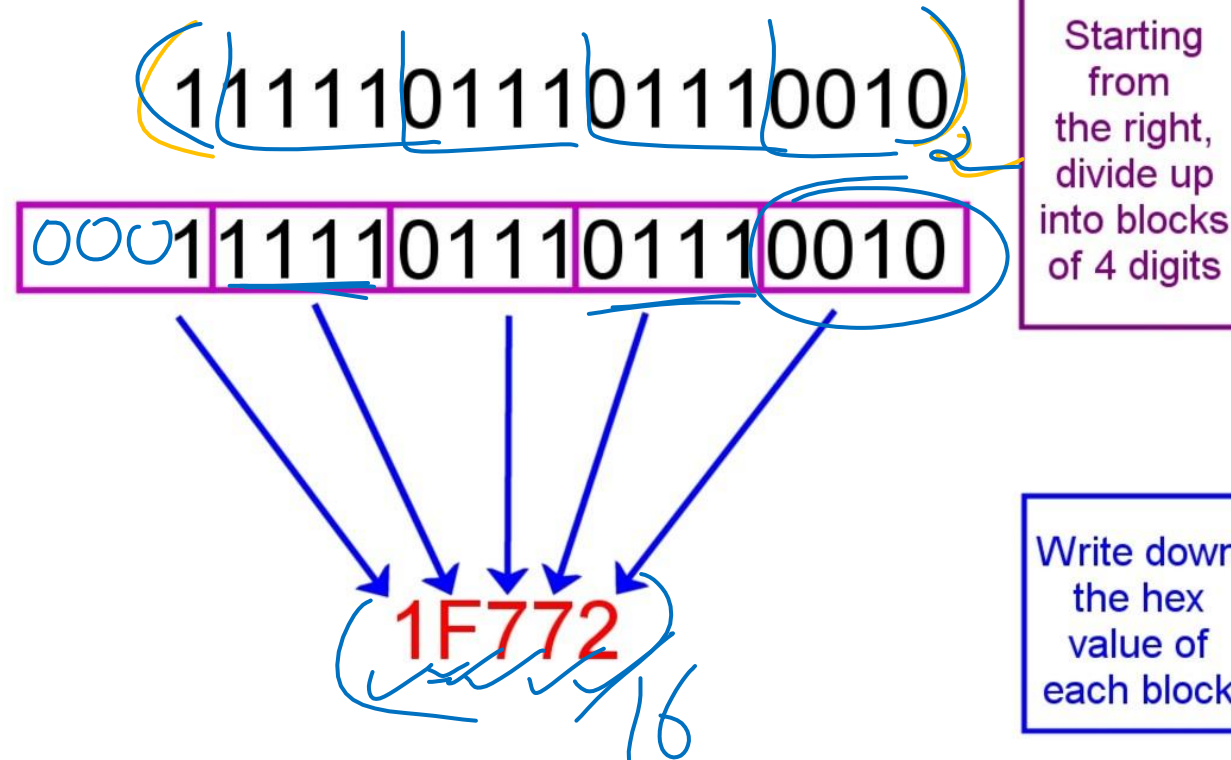
→ Right to Left

## Binary to Hexadecimal Table

| Binary<br>(Base 2) | Hexadecimal<br>(Base 16) | Binary<br>(Base 2) | Hexadecimal<br>(Base 16) |
|--------------------|--------------------------|--------------------|--------------------------|
| 0000               | 0                        | 1000               | 8                        |
| 0001               | 1                        | 1001               | 9                        |
| 0010               | 2                        | 1010               | A                        |
| 0011               | 3                        | 1011               | B                        |
| 0100               | 4                        | 1100               | C                        |
| 0101               | 5                        | 1101               | D                        |
| 0110               | 6                        | 1110               | E                        |
| 0111               | 7                        | 1111               | F                        |



# Converting Binary to Hex



$$(11111011101110010)_2 = (1F772)_{16}$$

$$(101101011)_2 = (0001\ 0110\ 1011)_2 = (16B)_{16}$$

# Octal and Hexadecimal to Binary Conversion

- Each octal digit is replaced by its 3-bit binary equivalent.
- Each hexadecimal digit is replaced by its 4-bit binary equivalent.

## Convert (345)<sub>8</sub> to Binary

$$\textcircled{3} \rightarrow \textcolor{blue}{011}, \textcircled{4} \rightarrow \textcolor{blue}{100}, \textcircled{5} \rightarrow \textcolor{green}{101}$$

Combining the groups:  $(345)_8 = \textcolor{blue}{(011100101)}_2$

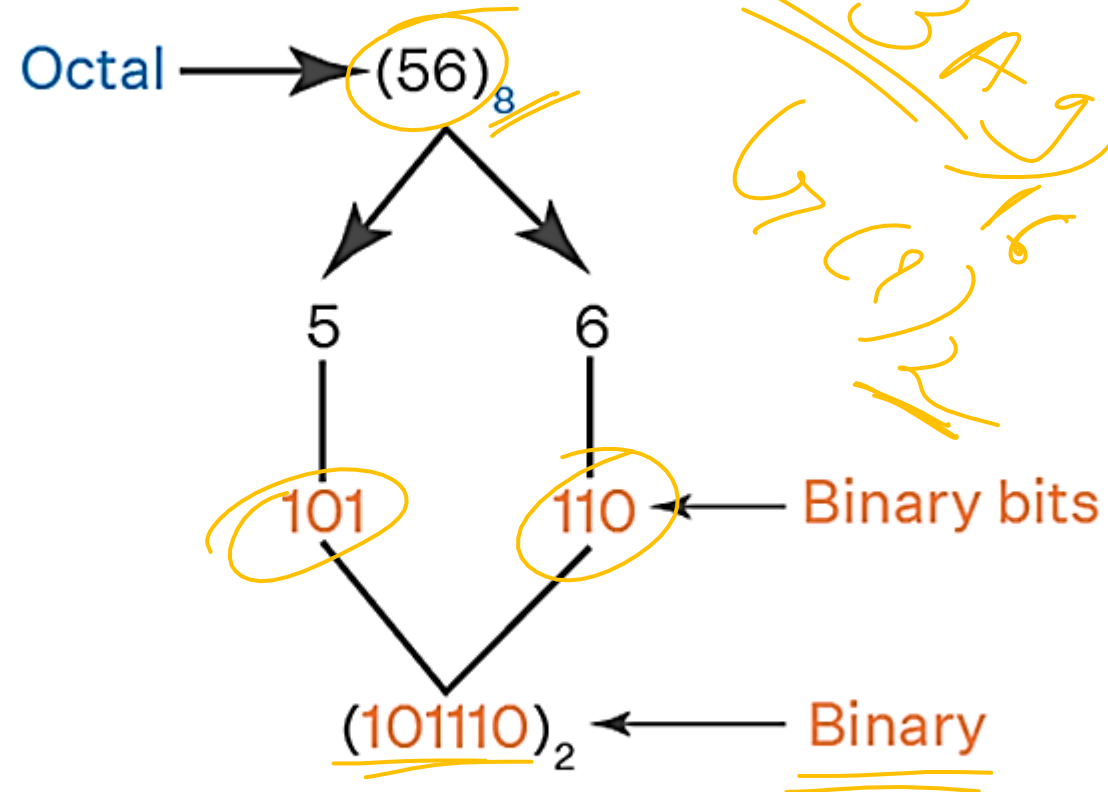
---

$$(3B2E)_{16} \Rightarrow (?)_2$$

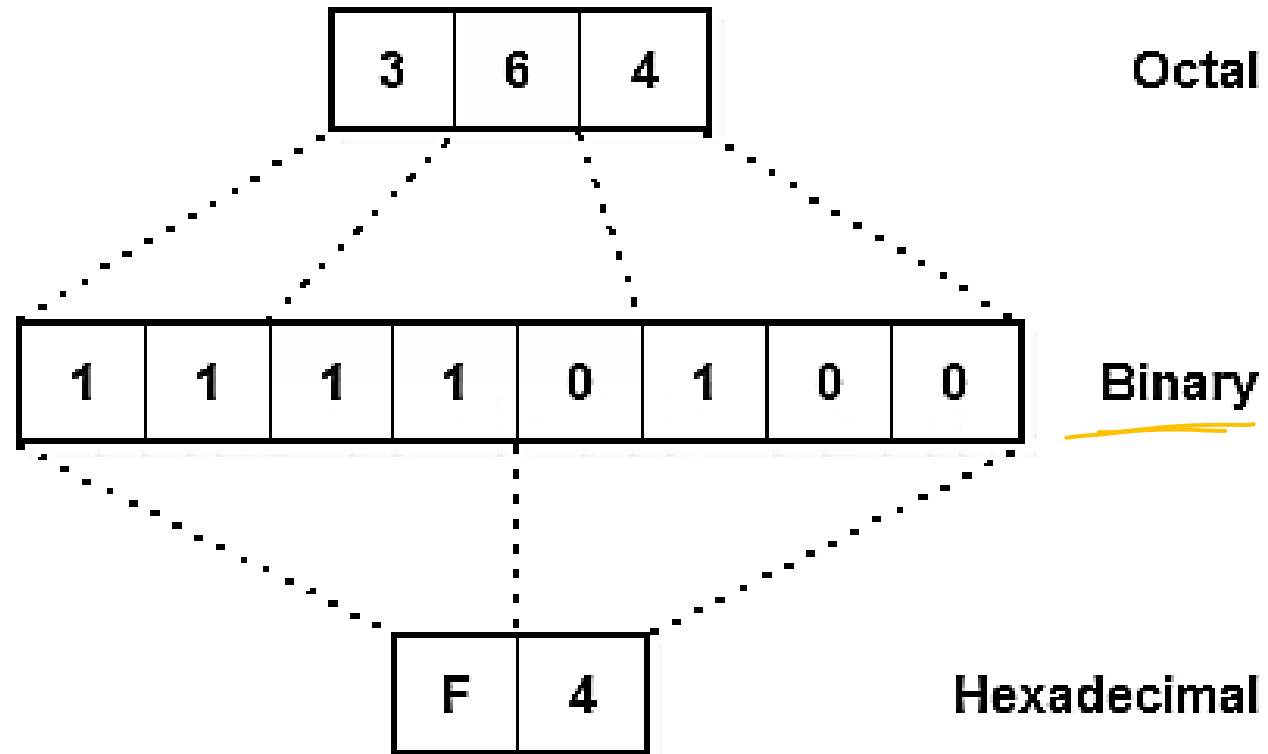
0011 1011 0010 1111

$$= (0011101100101111)_2$$

## Octal to Binary Conversion



$$\therefore (56)_8 = (101110)_2$$



# Boolean Logic

- While binary systems deal with the digits themselves, **Boolean Logic** is the mathematical framework for how those digits interact to make decisions.
- Named after 'George Boole'.
- It is the language of "True" and "False."
- In computing, we represent these as **1 (True)** and **0 (False)**.
- It forms the fundamental basis of digital electronics, computers, microprocessors, control units, and logic circuit design.

# Boolean Vs Binary

| <u>Aspect</u>      | Boolean Logic           | Binary System               |
|--------------------|-------------------------|-----------------------------|
| Nature             | Logical system          | Number system               |
| Purpose            | Decision-making & logic | Number representation       |
| Main operations    | AND, OR, NOT            | Addition, subtraction, etc. |
| Output meaning     | TRUE / FALSE            | Numerical value             |
| Mathematical basis | Boolean algebra         | Positional number system    |
| Use in electronics | Logic circuit design    | Data representation         |

# '+' Operation in Binary



- The result  $1 + 1 = 0$  with carry 1 happens because binary arithmetic uses base 2, not base 10.
- Decimal system (base 10): digits 0–9, When a sum reaches 10, we write 0 and carry 1.
- Binary system (base 2): digits 0 and 1 only, When a sum reaches 2, we must carry.

Boolean  $\Rightarrow + \Rightarrow 1 + 1 = 1$   
Decimal  $\Rightarrow 1 + 1 = 2$

- $1 + 1 = 2$  (in decimal), But 2 cannot be represented with a single binary digit, because binary has only 0 and 1.
- Now express 2 in binary:
- $(2)_{10} = (10)_2$  This means:
  - ✓ 0 is written in the current bit (sum)
  - ✓ 1 is carried to the next higher bit (carry)
- So,  $1 + 1 = (10)_2 \Rightarrow \text{Sum} = 0, \text{Carry} = 1.$

# Basic Concept of Boolean Logic

◆ In Boolean logic:

- ✓ Variables take binary values (**0 or 1**).
- ✓ Logical relationships are expressed using **Boolean expressions**.
- ✓ Operations are performed using **logic gates** implemented in hardware.

AND  
OR  
NOT } basic

# Three Primary Operators

| Operator                                 | Rule                                                        | Example                                                        |
|------------------------------------------|-------------------------------------------------------------|----------------------------------------------------------------|
| <b>AND ( · )</b>                         | Output is <b>True</b> only if <b>both</b> inputs are True.  | "I will go out if <b>it</b> is sunny <b>AND</b> I have money." |
| <b>OR ( + )</b>                          | Output is <b>True</b> if <b>at least one</b> input is True. | "I will wear a coat if it is raining <b>OR</b> it is snowing." |
| <b>NOT</b><br>( ~ or <sup>-</sup> or ' ) | Reverses the input (True becomes False, and vice versa).    | "I will eat if I am <b>NOT</b> full."                          |

# Truth Table

♦ A **truth table** shows all possible input combinations and the corresponding output.

→ for  $n$  inputs  $\Rightarrow 2^n$  combinations possible

| <u>Input A</u> | <u>Input B</u> | <u>A·B (AND)</u> | <u>A+B (OR)</u> | <u>A' (NOT)</u> | <u>B'</u> |
|----------------|----------------|------------------|-----------------|-----------------|-----------|
| 0              | 0              | 0                | 0               | 1               | 1         |
| 0              | 1              | 0                | 1               | 1               | 0         |
| 1              | 0              | 0                | 1               | 0               | 1         |
| 1              | 1              | 1                | 1               | 0               | 0         |

$2^2 \Rightarrow 4$  combinations for 2 I/Ps